

FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA

Arthur dos Santos Cabral, 566515, 2TDSA

Bruno Martins Bettio, 564939, 2TDSA

José Diogo da Silva Neves, 562341, 2TDSA

Júlia Tiziotto Buttler, 564975, 2TDSA

Mariana Xavier Quispe, 566357, 2TDSA

Code & Paws

São Paulo

2026

Sumário

1. Objetivo do Projeto.....	2
2. Escopo do Projeto.....	3
3. Tecnologias Utilizadas.....	3
4. Funcionalidades implementadas.....	4
Users.....	4
Pets.....	4
Diário de Entradas.....	4
Registros.....	5
5. Cronograma de desenvolvimento.....	5
6. Tabela de endpoints.....	5
7. Evidências de funcionamento.....	6
8. Diagramas.....	12
Diagrama de classe.....	12
Modelos de banco de dados.....	12
9. Links	13

1. Objetivo do Projeto

O objetivo do projeto é auxiliar no acompanhamento clínico e comportamental de pets, permitindo o registro contínuo de informações relevantes sobre a rotina e o estado de saúde dos animais.

A solução busca centralizar dados como alimentação, comportamento, sintomas, humor e observações gerais em uma estrutura organizada e persistente, facilitando o acompanhamento ao longo do tempo tanto pelos tutores quanto por profissionais veterinários.

Além das operações básicas de cadastro, o sistema foi desenvolvido com foco em organização de dados, relacionamentos entre entidades, validações, consultas estruturadas e persistência segura das informações, seguindo os princípios de Programação Orientada a Objetos e arquitetura RESTful.

2. Escopo do Projeto

O projeto contempla o desenvolvimento de uma API backend responsável pelo gerenciamento das principais entidades do sistema, incluindo usuários, pets, registros clínicos e diário de acompanhamento.

Entre as funcionalidades implementadas estão:

- cadastro, atualização, consulta e remoção de usuários;
- gerenciamento de pets vinculados aos tutores;
- registro diário de informações comportamentais;
- armazenamento estruturado de registros relacionados à rotina do pet;
- buscas por parâmetros específicos;
- listagens paginadas e ordenadas;
- validação de dados com Bean Validation;
- documentação automática da API com Swagger/OpenAPI;
- persistência de dados utilizando JPA/Hibernate e banco relacional H2.

3. Tecnologias Utilizadas

- Java 21
- Spring Boot
- Spring Data JPA
- Hibernate
- Maven
- H2 Database
- Bean Validation
- Swagger/OpenAPI
- Git e GitHub

4. Funcionalidades implementadas

Users

- cadastro;
- atualização;
- consulta;
- remoção;
- busca por e-mail;
- paginação.

Pets

- cadastro;
- relacionamento com usuário;
- atualização;
- consulta;
- remoção;
- paginação;
- busca por nome;
- busca por tutor.

Diário de Entradas

- criação de entradas diárias;
- busca por data;

- atualização;
- remoção;
- paginação.

Registros

- armazenamento de informações sobre o dia;
- atualização;
- consulta;
- remoção;
- paginação.

5. Cronograma de desenvolvimento

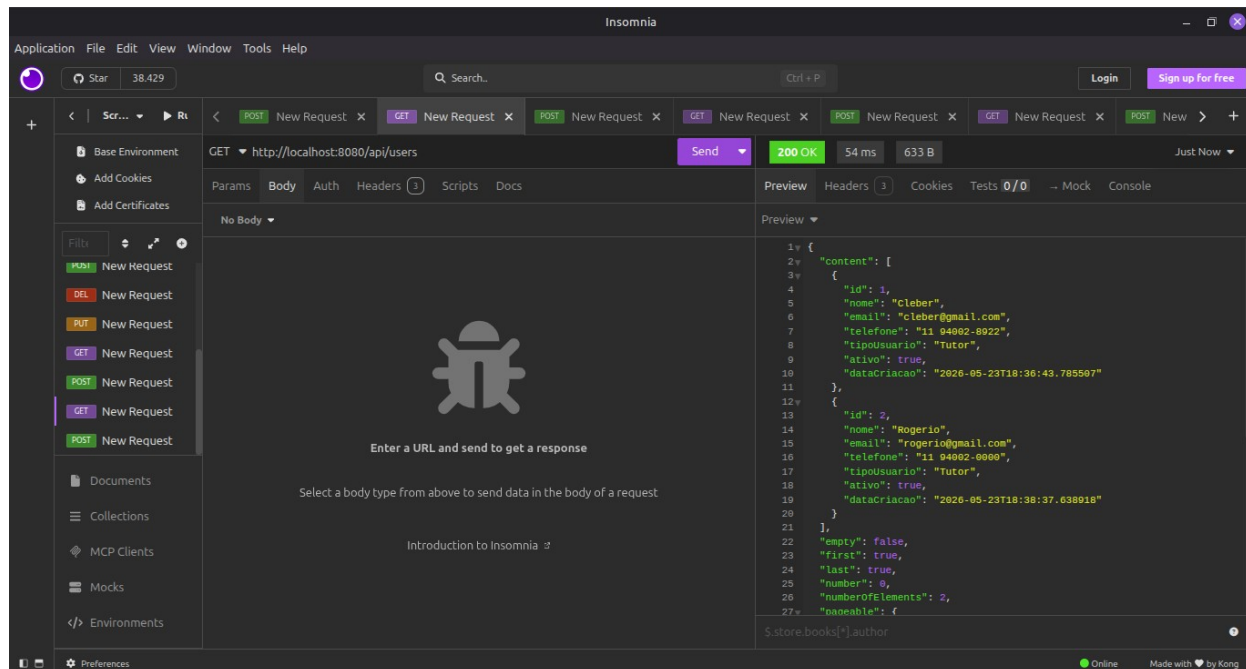
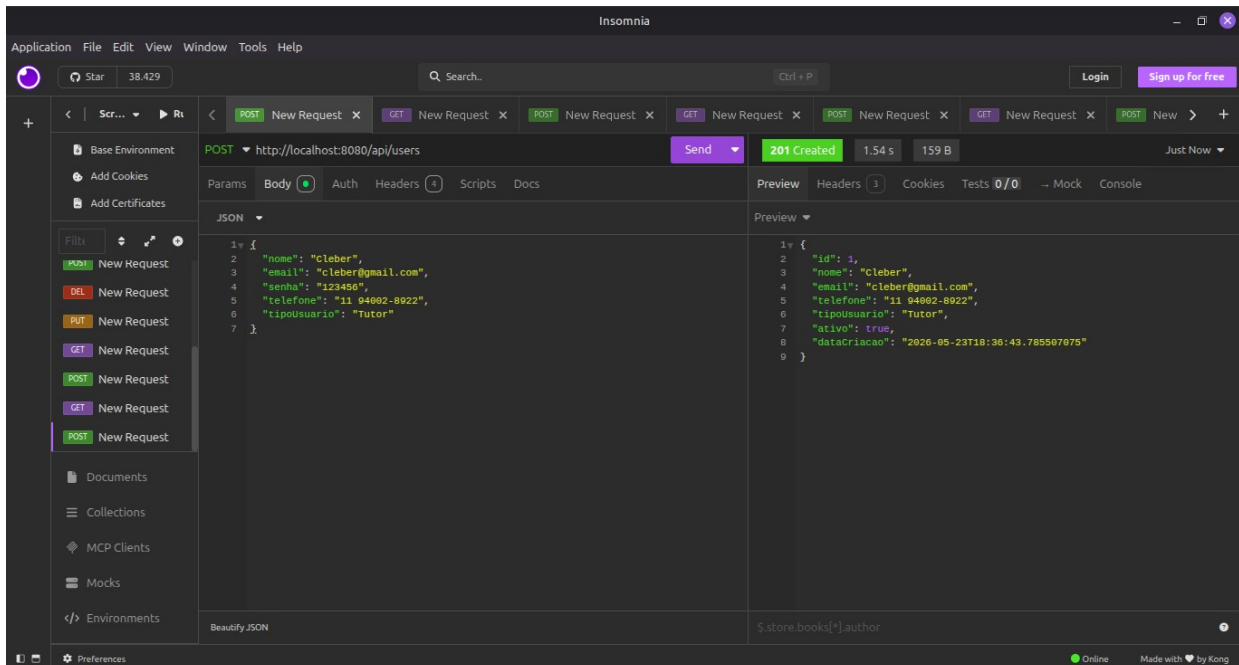
Atividade	Responsável	Período
Avaliação da proposta e planejamento	Equipe	Semana 1
Modelagem de dados	Júlia	Semana 1
Desenvolvimento do CRUD básico	Mariana	Semana 2
Revisão do andamento	Equipe	Semana 2
Implementação de camadas	Mariana	Semana 3
Documentação + revisão	Equipe	Semana 4

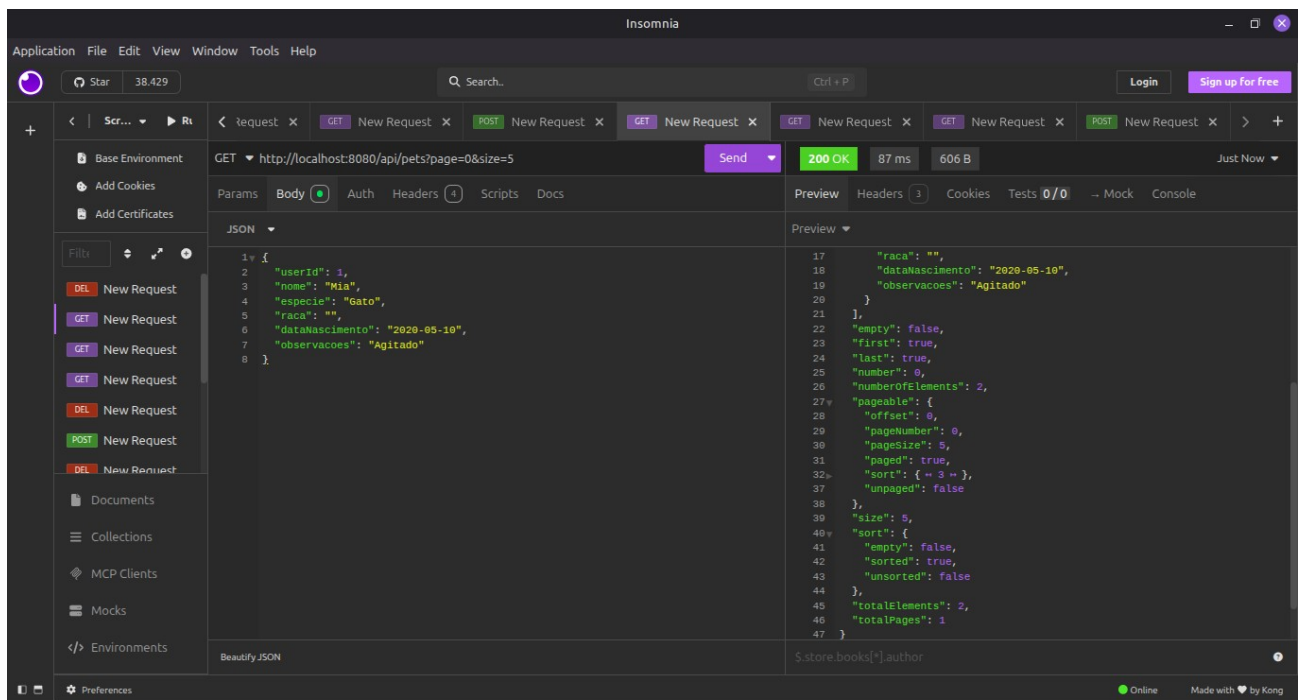
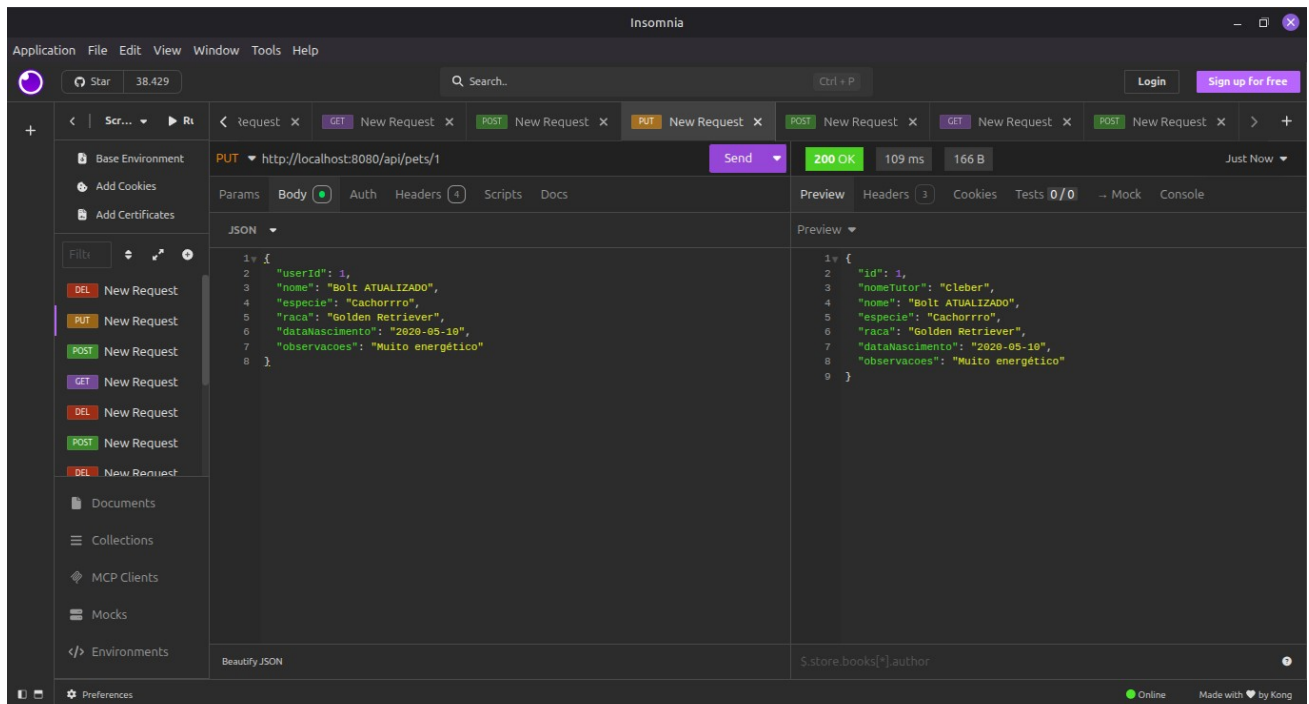
6. Tabela de endpoints

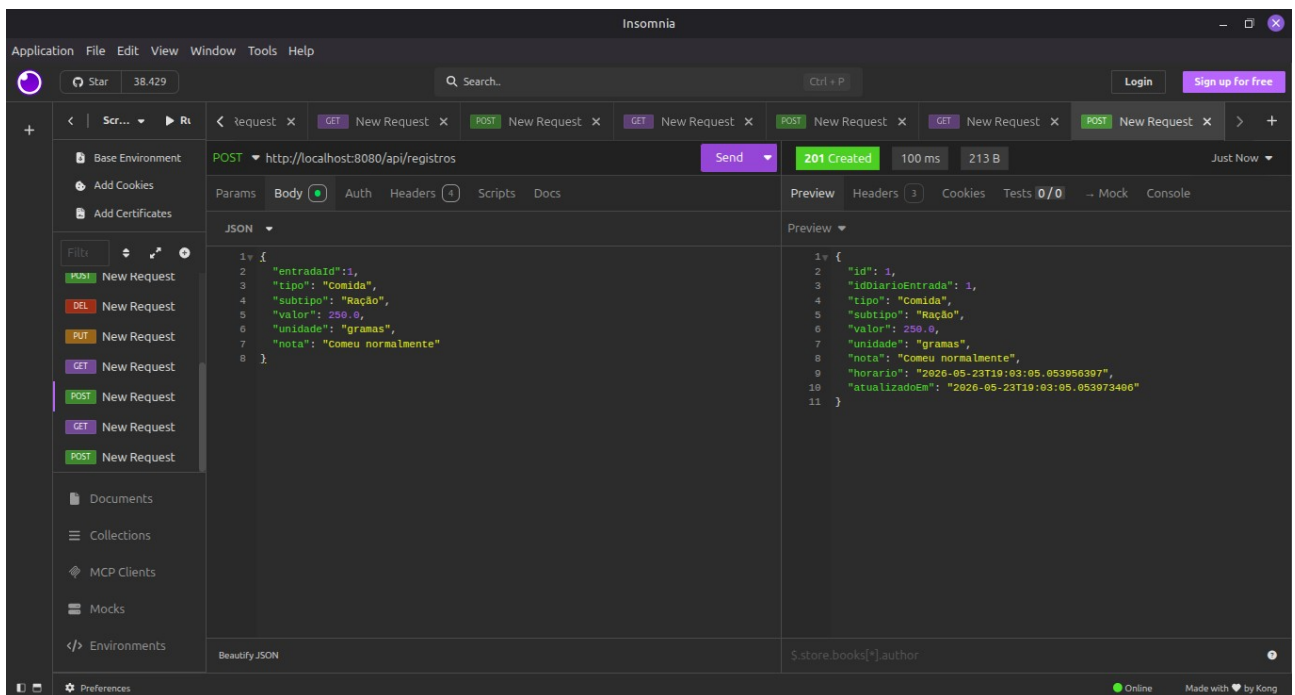
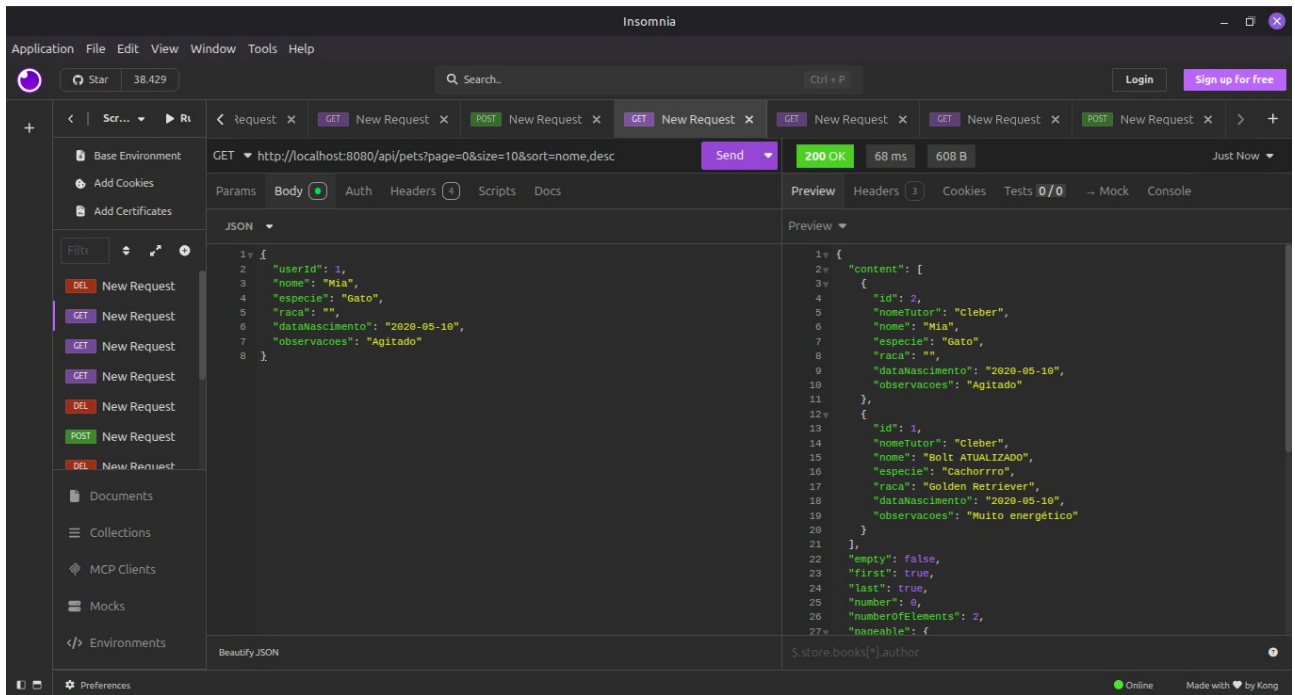
Endpoint	Método
/api/users	POST
/api/users	GET
/users/email/{email}	GET
/api/users?page=0&size=5	GET
/api/users/{id}	PUT
/api/users/{id}	DELETE
/api/pets	POST

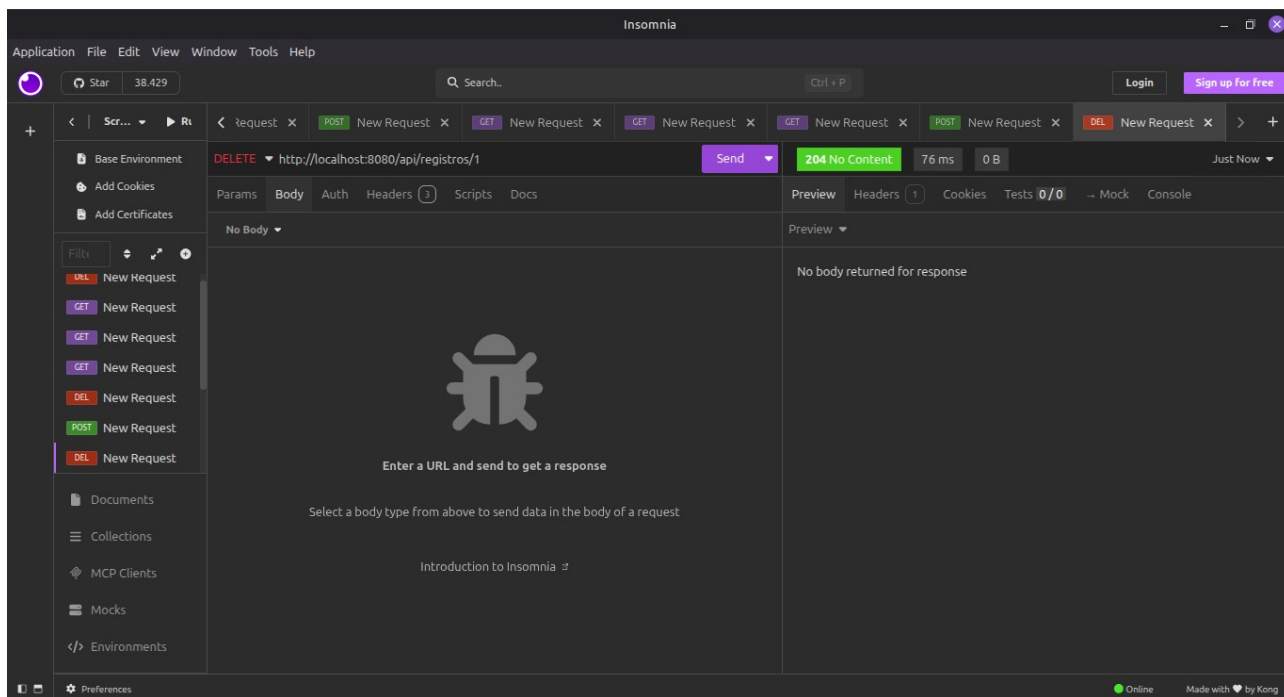
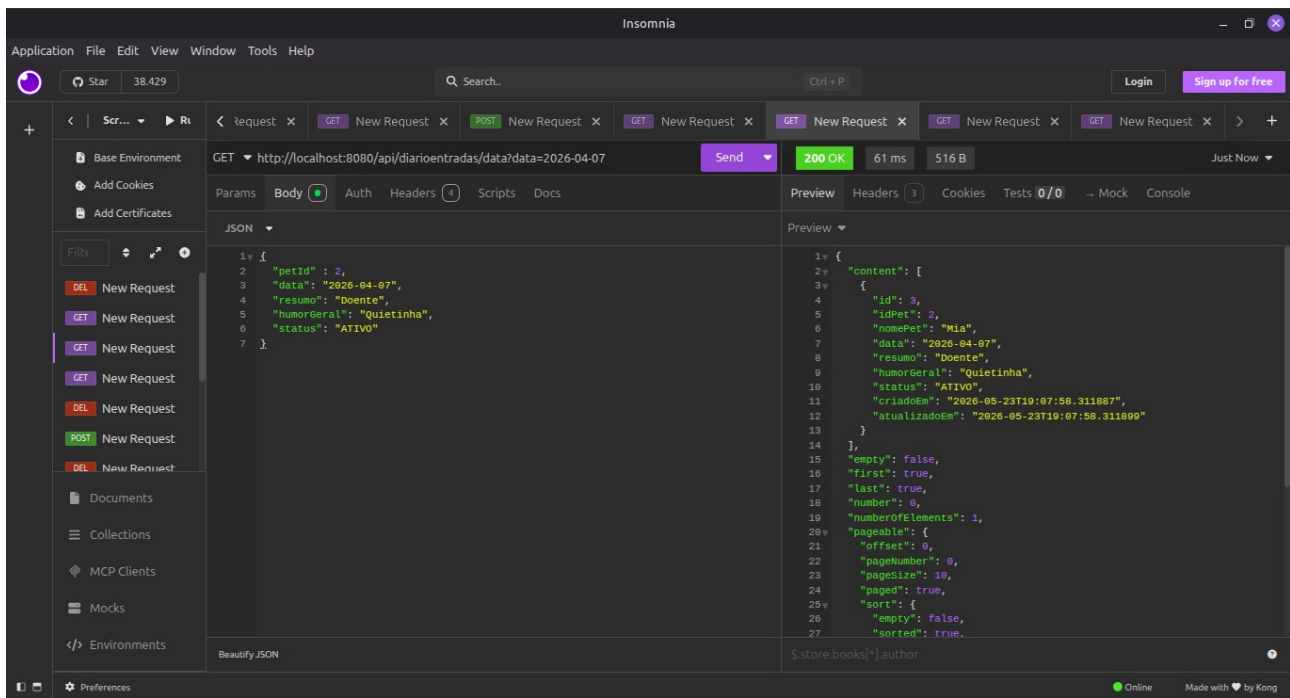
/api/pets/user/{id}	GET
/api/pets/nome/{nome}	GET
/api/pets?page=0&size=5	GET
/api/pets/{id}	PUT
/api/pets/{id}	DELETE
/api/diarioentradas	POST
/api/diarioentradas	GET
/api/diarioentradas/{id}	GET
/api/diarioentradas/data?data={data}	GET
/api/diarioentradas?page=0&size=5	GET
/api/diarioentradas/{id}	DELETE
/api/registros	POST
/api/registros	GET
/api/registros/{id}	GET
http://localhost:8080/api/registros?page=0&size=5	GET
/api/registros/{id}	PUT
/api/registros/{id}	DELETE

7. Evidências de funcionamento









POST

/api/users

Criar usuário

Cria um novo usuário no sistema.

Parameters

Cancel

No parameters

Request body

required

application/json

Edit Value

Schema

```
{
  "nome": "João Silva",
  "email": "joao@gmail.com",
  "senha": "123456",
  "telefone": "11 94002-8922",
  "tipoUsuario": "tutor"
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8080/api/users' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "nome": "João Silva",
    "email": "joao@gmail.com",
    "senha": "123456"
  }'
```

PUT

/api/users/{id}

Atualizar usuário

Atualiza os dados de um usuário existente.

Parameters

Cancel

Reset

Name

Description

id

required

integer(int64)

1

(path)

Request body

required

application/json

Edit Value

Schema

```
{
  "nome": "João Silva ATUALIZADO",
  "email": "joao@gmail.com",
  "senha": "123456",
  "telefone": "11 94002-8922",
  "tipoUsuario": "tutor"
}
```

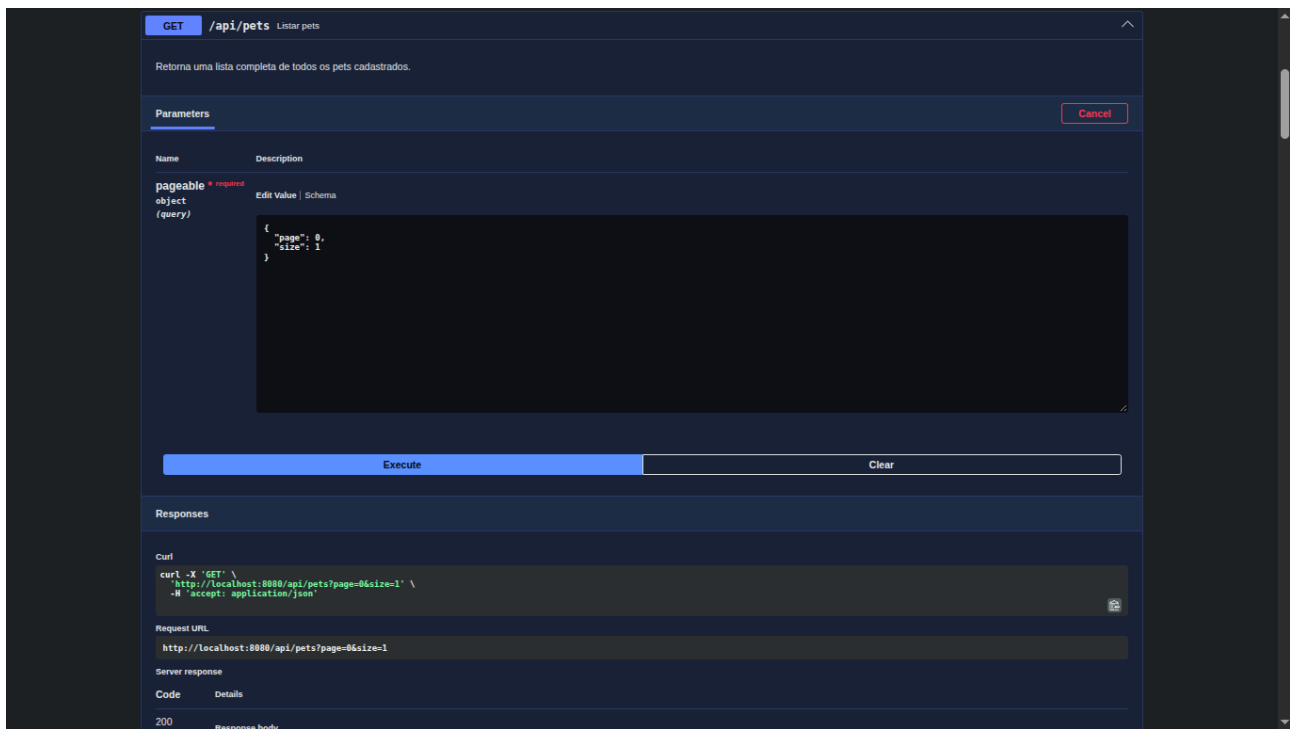
Execute

Clear

Responses

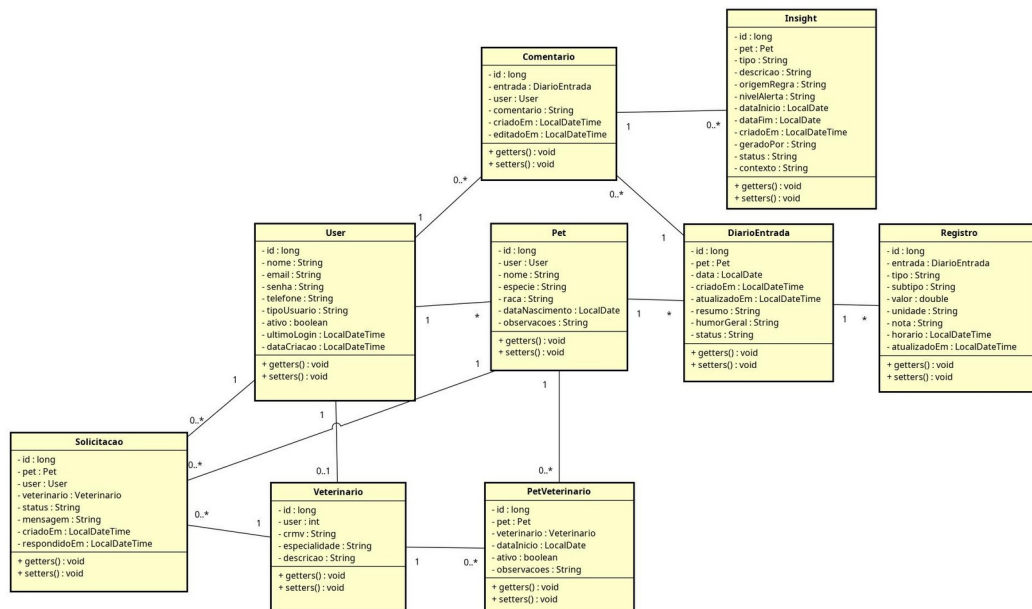
Curl

```
curl -X 'PUT' \
  'http://localhost:8080/api/users/1' \
  -H 'accept: */*' \
```

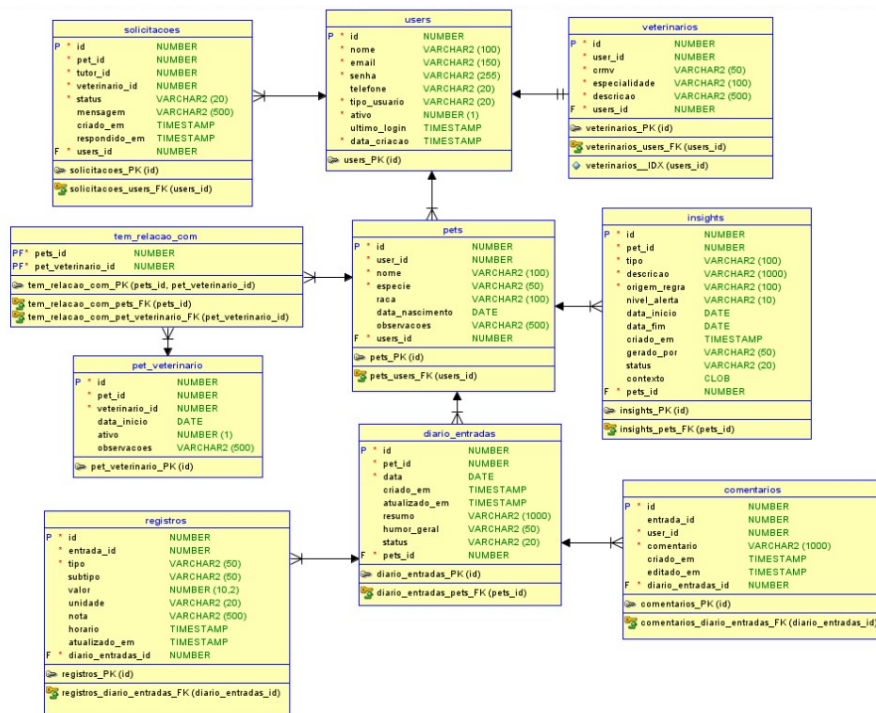
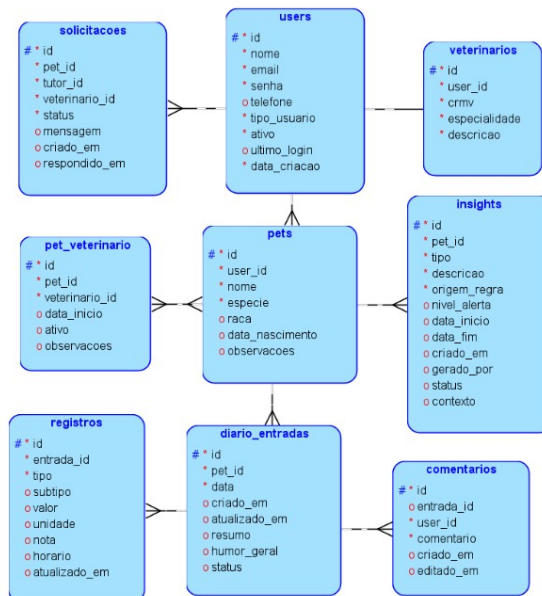


8. Diagramas

Diagrama de classe



Modelos de banco de dados



9. Links

Repositório no GitHub: <https://github.com/Marixavq/challenge-java-petcenter>

Render: <https://challenge-java-petcenter.onrender.com>

Swagger: <http://localhost:8080/swagger-ui/index.html>